

LAKCHE LLC

Chatwoot Fork & Customization Playbook

Fork Chatwoot Community Edition, rebrand it fully as LakcheLink, stage, and deploy to production on LakcheLink.com. Incorporates all v1.1 review corrections.

Version 1.1 — Revised

Product: LakcheLink.com · Base: Chatwoot Community Edition (MIT)

Audience: LakChe Engineering · Classification: Internal / Confidential

Prepared for the LakChe assigned software engineer

Document control

Field	Value
Document	LakcheLink Engineering Playbook — Fork, Rebrand & Deploy
Version	1.1 (supersedes 1.0)
Status	Approved for execution
Owner	LAKCHE — Founder & Lead Engineer, LakChe LLC
Date	01 July 2026
Classification	Internal / Confidential

What changed in v1.1 (review corrections)

#	Correction	Where
1	SuperAdmin account creation added — v1.0 only created an account admin, which would have locked you out of <code>/super_admin</code>	§9.7
2	Deployment templates now pulled from YOUR fork at the pinned tag — not from upstream <code>develop</code> , which could mismatch the deployed code	§9.2
3	Staging environment added as a mandatory phase before production	§8
4	Zero-typing secrets — every secret is generated and written by a single copy-paste block	§9.4
5	<code>/super_admin</code> lockdown — concrete Nginx snippets (IP allowlist or Basic Auth)	§9.6
6	VPS sizing guidance added	§3
7	Backup cron + tested restore procedure written out in full	§10.2
8	Trademark clarification — MIT covers copyright, not the Chatwoot name/logo	§1
9	Document-control block added to match LakChe governance house style	top

0. How to use this document

Work through the phases in order. Each phase has a "Definition of done." Do not skip Phase 1 (Licensing) — it determines what you may legally ship and how the image must be built.

Phase	What it covers	Est. effort
1	Licensing & legal gate (read first)	30 min reading
2	Architecture you're inheriting	30 min reading
3	Decisions to lock before coding	1 hr (with LAKCHE)
4	Fork & Git strategy	1–2 hrs
5	Local development environment	2–4 hrs
6	Rebranding to LakcheLink (file-by-file)	1–2 days
7	Functional customization (optional)	varies
8	Staging environment (deploy here first)	0.5 day
9	Production deployment on LakcheLink.com	1 day
10	Maintenance, backups & upstream updates	ongoing
11	Handoff checklist / acceptance criteria	—
A–D	Appendices (env vars, asset specs, pitfalls, cheat-sheet)	reference

Every command is written out in full and copy-paste-ready — zero manual typing required. All secrets are machine-generated by the provided blocks.

1. Licensing & legal gate — READ FIRST

What the license actually says

Chatwoot ships under a split license:

- **Everything outside the `enterprise/` directory is MIT ("MIT Expat").** MIT lets you use, modify, rebrand, self-host, and resell derivatives with essentially no restrictions. LakcheLink is built on this.
- **Everything inside `enterprise/` is under a separate Chatwoot Enterprise License** (not MIT). Distributing or running the Enterprise edition requires a paid subscription. (unless purchased)

Practical rules for LakcheLink

1. **Build the Community Edition only.** Use the CE build path / `-ce` image tags. Do not enable, depend on, or distribute features under `enterprise/` unless LakChe buys an Enterprise subscription.
2. **Rebranding via source code is explicitly permitted in CE.** The in-app white-label toggle is a *paid* feature — do not use it; edit the source/config instead (free and allowed).
3. **Keep the MIT notice.** Retain the `LICENSE` file and the `Copyright (c) Chatwoot Inc.` notice. Add LakChe's copyright for your own modifications; never remove Chatwoot's.
4. **Trademark ≠ copyright.** MIT grants a *copyright* license to the code — it grants **no rights to the "Chatwoot" name or logo**, which remain Chatwoot Inc.'s trademarks. Because we fully rename the product to **LakcheLink**, we inherently avoid trademark issues. The operational rule: never market LakcheLink using the Chatwoot name or logo, and never imply endorsement by Chatwoot Inc. A factual statement like "powered by open-source technology" is fine.

5. **Position LakcheLink as its own product.** Chatwoot encourages building niche platforms on CE (exactly what LakcheLink is) and discourages literal 1:1 clones/rebranding services of Chatwoot Cloud. Present LakcheLink as a LakChe product.

Action items

- [] Confirm with LAKCHE: **Community Edition only** (recommended) or budget for Enterprise.
- [] Verify no needed feature lives under `enterprise/`; if one does, drop it or license it.
- [] Retain `LICENSE` + Chatwoot copyright in the fork; product carries only LakcheLink branding.

Cleanest legal posture: base the build on the official **community-edition** path. Some teams also remove `enterprise/` from their fork before building — if you do, **test the build**; the codebase references enterprise modules conditionally. Verify the current CE build flag in `docker/Dockerfile` and the dev docs at your pinned tag.

2. Architecture you're inheriting

Component	Tech	Role
Web app	Ruby on Rails (Ruby 3.3.x)	API + server-rendered shell
Frontend	Vue.js built with Vite , package manager pnpm , Node 23	Dashboard SPA + embeddable chat widget
Database	PostgreSQL	Primary datastore
Cache / queue	Redis	Caching + Sidekiq queue + ActionCable backplane
Background jobs	Sidekiq	Async work (emails, webhooks, etc.)
Realtime	ActionCable (WebSockets at <code>/cable</code>)	Live conversation updates
Container images	<code>chatwoot/chatwoot</code> on Docker Hub	<code>latest</code> = full (incl. enterprise) · <code>latest-ce</code> = Community Edition ← use this

Branching model: Chatwoot uses git-flow; the active branch is `develop`. **Stable releases are tags (`vX.Y.Z`).** Pin LakcheLink to a stable tag, never to the moving `develop`.

Key branding files: `config/installation_config.yml` (name, logos, brand URLs, terms/privacy) · `public/brand-assets/` (`logo.svg` , `logo_dark.svg` , `logo_thumbnail.svg`) · `app/javascript/shared/store/globalConfig.js` · `app/views/layouts/vueapp.html.erb` (`<title>` , meta) · Tailwind config + CSS variables · mailer templates.

3. Phase 0 — Decisions to lock before coding

Decision	Options	Recommended default
Edition	CE (free, MIT) / Enterprise (paid)	CE
Deployment target	Single VPS w/ Docker / Kubernetes / PaaS	Single VPS + Docker Compose
VPS sizing	—	Minimum 2 vCPU / 4 GB RAM / 40 GB SSD for Rails+Sidekiq+Postgres+Redis. Recommended 4 vCPU / 8 GB once channels + the v2 AI service are live. Below 4 GB, asset precompile and Sidekiq will struggle.
Domain scheme	Root / subdomain	app.lakchelink.com (dashboard), staging.lakchelink.com (staging), root for marketing
Tenancy	Single account / multi-tenant	Multi-tenant if offered to clients (one isolated Account per client)
Public signup	Open / API-only / disabled	Disabled for v1 (invite-only)
Channels at launch	Widget, Email, WhatsApp, etc.	Website widget + Email first
Email provider	Resend / SES / SendGrid	Resend (already in LakChe stack)
File storage	Local disk / S3-compatible	S3-compatible (durable across rebuilds)
Brand assets	SVG logo light+dark, 512×512 mark	Prepare before Phase 6

4. Phase 1 — Fork & Git strategy

Own a fork you can brand **while still pulling future Chatwoot releases**. The trick: keep custom changes isolated so upstream merges stay clean.

4.1 Create the fork

Fork `https://github.com/chatwoot/chatwoot` into the LakChe org (e.g. `lakche/lakchelink`), then:

```
git clone git@github.com:lakche/lakchelink.git
cd lakchelink
git remote add upstream https://github.com/chatwoot/chatwoot.git
git fetch upstream --tags
```

4.2 Pin to a stable release and create the working branch

```
git tag --list 'v*' --sort=-version:refname | head -20
```

Pick the newest stable `vX.Y.Z`, then:

```
git checkout -b lakchelink-main vX.Y.Z
git push -u origin lakchelink-main
```

`lakchelink-main` is the long-lived branch: **the exact code that ships**, so anything copied from it (env template, compose file) always matches production.

4.3 Keep customizations isolated

Concentrate LakcheLink changes into few, predictable files: brand assets only under `public/brand-assets/` + `public/` favicons; branding config only in `config/installation_config.yml`; theme colors in Tailwind config + one CSS-variables block; custom logic in clearly named modules. One well-described commit per customization ("LakcheLink: brand assets", "LakcheLink: brand config", "LakcheLink: theme"). Upstream merges then conflict only in these files.

4.4 Secrets hygiene

Never commit real secrets; confirm `.env` is git-ignored. Commit a `.env.lakchelink.example` documenting variables with placeholders only.

Definition of done: fork exists with `upstream` remote · `lakchelink-main` created off a stable tag and pushed · no secrets committed.

5. Phase 2 — Local development environment

Run the unmodified app locally before changing anything, so branding bugs and setup bugs stay distinguishable.

Dev commands occasionally shift between versions — cross-check `Procfile.dev` in the repo and <https://developers.chatwoot.com/self-hosted> at your pinned tag.

Option A — Native (best for active development)

Prereqs: Ruby 3.3.x (rbenv/asdf), Node 23, pnpm, PostgreSQL, Redis, build tools.

```
gem install bundler
bundle install
pnpm install
cp .env.example .env
echo "SECRET_KEY_BASE=$(openssl rand -hex 64)" >> .env
bundle exec rails db:setup
overmind start -f Procfile.dev
# or: foreman start -f Procfile.dev
```

Open `http://localhost:3000`. Create the first dev admin via the Rails console:

```
bundle exec rails console
```

```

account = Account.create!(name: 'LakcheLink Dev')
user = User.new(name: 'Admin', email: 'admin@lakchelink.com', password: 'Password1!')
user.skip_confirmation!
user.save!
AccountUser.create!(account: account, user: user, role: :administrator)
SuperAdmin.create!(email: 'root@lakchelink.local', password: 'Password1!')
exit

```

Option B — Docker dev

```

cp .env.example .env
echo "SECRET_KEY_BASE=$(openssl rand -hex 64)" >> .env
docker compose -f docker-compose.yaml build
docker compose -f docker-compose.yaml run --rm rails bundle exec rails db:chatwoot_prepare
docker compose -f docker-compose.yaml up

```

Definition of done: unmodified Chatwoot runs locally; you can log in; hot-reload works (Option A).

6. Phase 3 — Rebrand to LakcheLink

Hard-coded values: `LakcheLink` (display), `lakchelink` (identifiers).

6.1 Replace brand assets (keep exact filenames)

File	Used for	Spec
<code>public/brand-assets/logo.svg</code>	Dashboard + login (light)	SVG horizontal lockup
<code>public/brand-assets/logo_dark.svg</code>	Dark mode	Light-on-dark variant
<code>public/brand-assets/logo_thumbnail.svg</code>	Favicon / app icon source	512×512 square mark

Also replace `public/favicon.ico` and any apple-touch/PWA icons.

6.2 Install-wide branding config

Edit `config/installation_config.yml` — change the values of these existing keys:

```

- name: INSTALLATION_NAME
  value: 'LakcheLink'
- name: LOGO_THUMBNAIL
  value: '/brand-assets/logo_thumbnail.svg'
- name: LOGO
  value: '/brand-assets/logo.svg'
- name: LOGO_DARK
  value: '/brand-assets/logo_dark.svg'
- name: BRAND_NAME
  value: 'LakcheLink'
- name: BRAND_URL
  value: 'https://www.lakchelink.com'
- name: WIDGET_BRAND_URL
  value: 'https://www.lakchelink.com'
- name: TERMS_URL
  value: 'https://www.lakchelink.com/terms'
- name: PRIVACY_URL
  value: 'https://www.lakchelink.com/privacy'

```

`BRAND_NAME` / `BRAND_URL` drive the "Powered by ..." text in emails and the widget. Leave `locked: true` defaults so the values are authoritative.

Applying: fresh DBs load the YAML during `rails db:chatwoot_prepare`. Existing DBs are **NOT** overwritten (seeds are find-or-create) — update rows via console, then **restart web + Sidekiq** to clear the config cache:

```

%w[INSTALLATION_NAME BRAND_NAME].each do |k|
  InstallationConfig.find_by(name: k)&.update!(value: 'LakcheLink')
end
InstallationConfig.find_by(name: 'BRAND_URL')&.update!(value: 'https://www.lakchelink.com')
InstallationConfig.find_by(name: 'WIDGET_BRAND_URL')&.update!(value: 'https://
www.lakchelink.com')

```

6.3 Remove "Powered by Chatwoot" in widget + emails

CE path = edit source (the UI toggle is the paid feature). Widget: locate the branding component under `app/javascript/widget/` (grep `branding`, `Powered`, `WIDGET_BRAND`) and point it at LakcheLink or remove the block. Emails: mailer layouts under `app/views/` follow `BRAND_NAME` / `BRAND_URL` automatically once set.

```

grep -ri "powered by" app/ | grep -i chatwoot
grep -ri "chatwoot" public/brand-assets app/views/layouts

```

6.4 Theme colors

Update Tailwind theme tokens + the `:root` CSS variables palette to LakcheLink colors. Widget color is *also* per-inbox: Inbox → Configuration → Widget Color.

6.5 App shell, title, PWA manifest

`app/views/layouts/vueapp.html.erb`: `<title>`, meta description, any hard-coded "Chatwoot". Update the PWA/web manifest (name, short name, theme color, icons).

6.6 Sender identity

`MAILER_SENDER_EMAIL=LakcheLink Support <support@lakchelink.com>` (Appendix A).

6.7 Rebuild & verify

Restart dev server, hard-refresh, walk: login page, dashboard header, tab favicon+title, a test widget, a password-reset email. **Zero "Chatwoot" visible to end users.**

Definition of done: all checkmarks above pass on a fresh install.

7. Phase 4 — Functional customization (optional)

- **Signup control:** `ENABLE_ACCOUNT_SIGNUP=false` (invite-only) for launch.
- **Channels:** enable only launch channels; add inbox-by-inbox.
- **Custom attributes & teams:** model the LakcheLink workflow in the dashboard (no code).
- **Multi-tenancy:** one Account per client = full isolation out of the box.
- **Widget SDK:** embed LakcheLink chat into lakche.com, HibisXpert, KonectX with identified-user context.

8. Phase 5 — Staging environment (deploy here first)

Rule: nothing reaches production that hasn't run on staging — every upstream merge, every rebrand change, and (in v2) every alpha/beta channel. Staging is one extra compose project and one subdomain.

8.1 Provision

On the same VPS (if ≥ 8 GB RAM) or a small separate one:

```
sudo mkdir -p /opt/lakchelink-staging && cd /opt/lakchelink-staging
git clone --depth 1 --branch lakchelink-main git@github.com:lakche/lakchelink.git /tmp/
lakchelink-src
cp /tmp/lakchelink-src/.env.example .env
cp /tmp/lakchelink-src/docker-compose.production.yaml docker-compose.yaml
rm -rf /tmp/lakchelink-src
```

Point the compose file's `rails` and `sidekiq` services at `ghcr.io/lakche/lakchelink:staging` (or reuse `:latest` pre-promotion), bind rails to `127.0.0.1:3001:3000`, and generate secrets with the same zero-typing block as §9.4 (set `FRONTEND_URL=https://staging.lakchelink.com`). Use a **separate database name** (`lakchelink_staging`) and separate storage bucket/prefix.

8.2 Expose

DNS A record `staging.lakchelink.com` → VPS. Duplicate the Nginx block from §9.6 with `server_name staging.lakchelink.com;` and `proxy_pass http://127.0.0.1:3001;`, then:

```
sudo certbot --nginx -d staging.lakchelink.com
```

8.3 Promotion flow

```
merge upstream tag → build image → deploy to STAGING → run $11 smoke checks → tag
image :production → deploy to PRODUCTION
```

Definition of done: staging serves over HTTPS on its own DB; the team can log in; a change deployed to staging does not affect production.

9. Phase 6 — Production deployment on LakcheLink.com

Stack: **Ubuntu VPS + Docker Compose + Nginx + Let's Encrypt**, running **your own built image**.

9.1 Build and publish the LakcheLink image

```
# from the repo root on lakchelink-main
docker build -f docker/Dockerfile -t ghcr.io/lakche/lakchelink:latest .
echo "$GHCR_TOKEN" | docker login ghcr.io -u <github-username> --password-stdin
docker push ghcr.io/lakche/lakchelink:latest
```

Automate later in CI (GitHub Actions: build on push to `lakchelink-main`, tag with SHA + `latest`).

9.2 Provision the server + fetch templates FROM YOUR FORK (v1.1 fix)

```
# Docker Engine + Compose plugin
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh
sudo apt-get update && sudo apt-get install -y docker-compose-plugin
docker --version && docker compose version
```

```
# templates come from the SAME branch/tag the image was built from – never from upstream
develop
sudo mkdir -p /opt/lakchelink && cd /opt/lakchelink
git clone --depth 1 --branch lakchelink-main git@github.com:lakche/lakchelink.git /tmp/
lakchelink-src
sudo cp /tmp/lakchelink-src/.env.example /opt/lakchelink/.env
sudo cp /tmp/lakchelink-src/docker-compose.production.yaml /opt/lakchelink/docker-compose.yaml
rm -rf /tmp/lakchelink-src
```

If the fork is private, add the server's SSH key as a read-only **deploy key** on the repo first (Repo → Settings → Deploy keys).

9.3 Point compose at your image

Edit `docker-compose.yaml`: for **both** `rails` and `sidekiq` services replace `image: chatwoot/chatwoot:latest` with:

```
image: ghcr.io/lakche/lakchelink:latest
```

9.4 Zero-typing secret generation + environment (v1.1 fix)

One block generates every secret, writes them into `.env`, and prints a copy for the password manager:

```

cd /opt/lakchelink
SECRET_KEY_BASE=$(openssl rand -hex 64)
POSTGRES_PASSWORD=$(openssl rand -base64 24 | tr -dc 'a-zA-Z0-9' | head -c 24)
REDIS_PASSWORD=$(openssl rand -base64 24 | tr -dc 'a-zA-Z0-9' | head -c 24)
ADMIN_PASSWORD=$(openssl rand -base64 18 | tr -dc 'a-zA-Z0-9' | head -c 16)
SUPERADMIN_PASSWORD=$(openssl rand -base64 18 | tr -dc 'a-zA-Z0-9' | head -c 16)

cat >> .env << EOF

# --- LakcheLink production (generated $(date +%F)) ---
SECRET_KEY_BASE=$SECRET_KEY_BASE
FRONTEND_URL=https://app.lakchelink.com
RAILS_ENV=production
NODE_ENV=production
INSTALLATION_ENV=docker
DEFAULT_LOCALE=en
ENABLE_ACCOUNT_SIGNUP=false
POSTGRES_HOST=postgres
POSTGRES_PORT=5432
POSTGRES_DATABASE=lakchelink_production
POSTGRES_USERNAME=lakchelink
POSTGRES_PASSWORD=$POSTGRES_PASSWORD
REDIS_URL=redis://redis:6379
REDIS_PASSWORD=$REDIS_PASSWORD
MAILER_SENDER_EMAIL=LakcheLink Support <support@lakchelink.com>
SMTP_ADDRESS=smtp.resend.com
SMTP_PORT=465
SMTP_USERNAME=resend
SMTP_AUTHENTICATION=login
SMTP_ENABLE_STARTTLS_AUTO=true
SMTP_DOMAIN=lakchelink.com
ACTIVE_STORAGE_SERVICE=s3_compatible
EOF

echo "=====
echo "SAVE THESE IN THE PASSWORD MANAGER NOW:"
echo "POSTGRES_PASSWORD=$POSTGRES_PASSWORD"
echo "REDIS_PASSWORD=$REDIS_PASSWORD"
echo "ADMIN_PASSWORD=$ADMIN_PASSWORD          (dashboard admin - used in 9.7)"
echo "SUPERADMIN_PASSWORD=$SUPERADMIN_PASSWORD (super_admin panel - used in 9.7)"
echo "=====

```

Two things still need pasting (not typing): the **Resend API key** as `SMTP_PASSWORD=` and the **S3 credentials** (`S3_BUCKET_NAME`, `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, `AWS_REGION`) — append them with `nano .env` or:

```

cat >> .env << 'EOF'
SMTP_PASSWORD=PASTE_RESEND_KEY_HERE
S3_BUCKET_NAME=PASTE_BUCKET
AWS_ACCESS_KEY_ID=PASTE_KEY
AWS_SECRET_ACCESS_KEY=PASTE_SECRET
AWS_REGION=PASTE_REGION
EOF

```

Also mirror `POSTGRES_PASSWORD` / `REDIS_PASSWORD` into `docker-compose.yml` where the postgres/redis services define them (open the file; the generated values are printed above for pasting).

Verify the full variable list against `.env.example` in your fork at the pinned tag — names shift between versions.

9.5 Initialize and start

```
docker compose run --rm rails bundle exec rails db:chatwoot_prepare
docker compose up -d
curl -I localhost:3000/api      # expect HTTP/1.1 200
```

9.6 DNS + Nginx + SSL + /super_admin lockdown (v1.1 fix)

DNS: A record `app.lakchelink.com` → VPS IP.

```
# /etc/nginx/sites-available/lakchelink
server {
    listen 80;
    server_name app.lakchelink.com;
    location / { return 301 https://$server_name$request_uri; }
}

server {
    listen 443 ssl http2;
    server_name app.lakchelink.com;

    ssl_certificate      /etc/letsencrypt/live/app.lakchelink.com/fullchain.pem;
    ssl_certificate_key  /etc/letsencrypt/live/app.lakchelink.com/privkey.pem;
    ssl_protocols TLSv1.2 TLSv1.3;

    add_header X-Frame-Options "SAMEORIGIN" always;
    add_header X-Content-Type-Options "nosniff" always;
    client_max_body_size 50M;

    # ---- Super Admin panel: locked down ----
    # Option 1 (preferred if LakChe has a static IP): allowlist
    location /super_admin {
        allow 203.0.113.10;      # replace with LakChe office/VPN IP (paste, digits only)
        deny all;
        proxy_pass http://127.0.0.1:3000;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    location / {
        proxy_pass http://127.0.0.1:3000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_read_timeout 86400;    # WebSockets for /cable – do not remove
    }
}
```

Option 2 — no static IP? Use Basic Auth instead of the allow/deny pair:

```
sudo apt-get install -y apache2-utils
sudo htpasswd -c /etc/nginx/.htpasswd_superadmin lakche_root
```

```
location /super_admin {
    auth_basic "Restricted";
    auth_basic_user_file /etc/nginx/.htpasswd_superadmin;
    proxy_pass http://127.0.0.1:3000;
    proxy_http_version 1.1;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
    proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto $scheme;
}
```

Enable + certify:

```
sudo ln -s /etc/nginx/sites-available/lakchelink /etc/nginx/sites-enabled/lakchelink
sudo nginx -t
sudo systemctl reload nginx
sudo certbot --nginx -d app.lakchelink.com
```

9.7 First admin AND SuperAdmin (v1.1 fix — both are required)

Two different things: the **account administrator** runs the dashboard; the **SuperAdmin** is a separate record that unlocks the `/super_admin` installation console (feature flags, accounts, and — in v2 — Captain/TikTok toggles). v1.0 created only the first; without the second you are locked out of the panel.

```
docker compose exec rails bundle exec rails console
```

```
# 1) Dashboard account + admin (paste ADMIN_PASSWORD from 9.4)
account = Account.create!(name: 'LakcheLink')
user = User.new(name: 'Admin', email: 'admin@lakchelink.com', password: 'PASTE_ADMIN_PASSWORD')
user.skip_confirmation!
user.save!
AccountUser.create!(account: account, user: user, role: :administrator)

# 2) Installation SuperAdmin (paste SUPERADMIN_PASSWORD from 9.4)
SuperAdmin.create!(email: 'root@lakchelink.com', password: 'PASTE_SUPERADMIN_PASSWORD')
exit
```

Verify: https://app.lakchelink.com/super_admin accepts the SuperAdmin login (through the Nginx lock).

Definition of done: image from fork deployed · `/api` returns 200 · HTTPS valid · WebSocket live updates verified (two browsers) · admin + SuperAdmin both log in · `/super_admin` blocked from non-allowed sources · test email sends via Resend.

10. Phase 7 — Maintenance, backups & upstream updates

10.1 Pull a new Chatwoot release

```
git fetch upstream --tags
git checkout lakchelink-main
git merge vX.Y.Z # new stable tag; conflicts confined to branded files
git push origin lakchelink-main
```

Rebuild + push image (§9.1), **deploy to staging first (§8.3)**, run the §11 smoke checks, then on production:

```
cd /opt/lakchelink
docker compose pull
docker compose run --rm rails bundle exec rails db:chatwoot_prepare
docker compose up -d
```

Many versions behind? Upgrade **iteratively through intermediate tags**, reading each release's notes and running `db:chatwoot_prepare` after each step.

10.2 Backups — cron + tested restore (v1.1 fix)

Nightly dump (with `--clean --if-exists` so restores are one command), 14-day retention:

```
sudo mkdir -p /var/backups/lakchelink
( sudo crontab -l 2>/dev/null; echo '0 2 * * * cd /opt/lakchelink && docker compose exec -T
postgres pg_dump -U lakchelink --clean --if-exists lakchelink_production | gzip > /var/backups/
lakchelink/db_$(date +%F).sql.gz && find /var/backups/lakchelink -name "db_*.sql.gz" -mtime
+14 -delete' ) | sudo crontab -
```

Ship the dumps off-box (rclone/S3 sync) — a backup on the same disk is not a backup.

Restore procedure (run it once now as a drill, on staging):

```
cd /opt/lakchelink
docker compose stop rails sidekiq
gunzip -c /var/backups/lakchelink/db_YYYY-MM-DD.sql.gz | docker compose exec -T postgres psql -
U lakchelink -d lakchelink_production
docker compose start rails sidekiq
curl -I localhost:3000/api # expect 200
```

Uploads live in S3-compatible storage (already durable) — enable bucket versioning as well.

10.3 Security

Rotate `SECRET_KEY_BASE`, DB and Redis credentials per LakChe policy · keep host + base image patched; rebuild on upstream security releases · watch Chatwoot `SECURITY.md` /advisories · `/super_admin` and SSH stay locked down.

11. Handoff checklist (acceptance criteria)

- [] **CE only**; `LICENSE` + Chatwoot copyright retained; no enterprise code shipped; no Chatwoot name/logo in LakcheLink marketing.
- [] Fork under LakChe org, pinned to a stable tag, `upstream` configured, customizations isolated on `lakchelink-main`.
- [] **Zero** visible "Chatwoot" anywhere an end user looks (login, dashboard, tab/favicon, widget, emails).
- [] **Staging** live at `staging.lakchelink.com` on its own DB; promotion flow documented and used.
- [] Production at `https://app.lakchelink.com`: HTTPS valid · WebSockets verified · Resend email verified.
- [] Account admin **and** SuperAdmin exist; `/super_admin` reachable only via allowlist/Basic Auth.
- [] Backup cron installed; **restore drill performed once** on staging and documented.

- [] Runbook handed back: build/push image, staging → production promotion, restore procedure.

Appendix A — Production environment variables (reference)

The §9.4 block generates/writes the core set. Full reference (verify names against `.env.example` in your fork at the pinned tag):

```
SECRET_KEY_BASE=<generated>
FRONTEND_URL=https://app.lakchelink.com
RAILS_ENV=production
NODE_ENV=production
INSTALLATION_ENV=docker
DEFAULT_LOCALE=en
RAILS_MAX_THREADS=5
ENABLE_ACCOUNT_SIGNUP=false
POSTGRES_HOST=postgres
POSTGRES_PORT=5432
POSTGRES_DATABASE=lakchelink_production
POSTGRES_USERNAME=lakchelink
POSTGRES_PASSWORD=<generated>
REDIS_URL=redis://redis:6379
REDIS_PASSWORD=<generated>
MAILER_SENDER_EMAIL=LakcheLink Support <support@lakchelink.com>
SMTP_ADDRESS=smtp.resend.com
SMTP_PORT=465
SMTP_USERNAME=resend
SMTP_PASSWORD=<paste Resend API key>
SMTP_AUTHENTICATION=login
SMTP_ENABLE_STARTTLS_AUTO=true
SMTP_DOMAIN=lakchelink.com
ACTIVE_STORAGE_SERVICE=s3_compatible
S3_BUCKET_NAME=<paste>
AWS_ACCESS_KEY_ID=<paste>
AWS_SECRET_ACCESS_KEY=<paste>
AWS_REGION=<paste>
```

Appendix B — Brand asset specification

Asset	Path	Format	Notes
Logo (light)	<code>public/brand-assets/logo.svg</code>	SVG	Horizontal lockup, transparent bg
Logo (dark)	<code>public/brand-assets/logo_dark.svg</code>	SVG	Light-on-dark
App mark	<code>public/brand-assets/logo_thumbnail.svg</code>	SVG	512×512 square
Favicon	<code>public/favicon.ico</code>	ICO	Multi-resolution
PWA/touch icons	<code>public/</code> per manifest	PNG	192×192 + 512×512

Keep filenames identical — only contents change.

Appendix C — Common pitfalls & fixes

Symptom	Cause	Fix
Branding doesn't show on existing install	Seeds are find-or-create	Console update (§6.2) + restart rails & sidekiq
Old logo persists after edit	In-memory config cache	<code>docker compose restart rails sidekiq</code>
No realtime updates	Nginx not upgrading <code>/cable</code> WebSockets	Keep <code>Upgrade / Connection "upgrade"</code> headers (§9.6)
<code>db:migrate</code> errors on first install	Wrong task	Use <code>rails db:chatwoot_prepare</code>
Can't log into <code>/super_admin</code>	No SuperAdmin record (v1.0 bug)	§9.7 step 2
<code>.env</code> var mismatch on deploy	Templates from upstream <code>develop</code>	Pull templates from your fork (§9.2)
Emails not sending	SMTP misconfig	Recheck Resend creds; verified sending domain; test from console
Assets 404 in production	Wrong image / no precompile	Deploy the LakcheLink image (Dockerfile runs precompile)
Duplicate-key on installation config	Pre-existing locked row	Update the row, don't insert

Appendix D — Operations cheat-sheet

```
docker compose logs -f rails
docker compose logs -f sidekiq
docker compose restart rails sidekiq
docker compose exec rails bundle exec rails console
docker compose run --rm rails bundle exec rails db:chatwoot_prepare
docker compose pull && docker compose up -d
docker compose exec -T postgres pg_dump -U lakchelink --clean --if-exists lakchelink_production
| gzip > lakchelink_$(date +%F).sql.gz
```

End of Playbook v1.1 — build CE-only, keep customizations isolated, stage before production, and drill the restore.